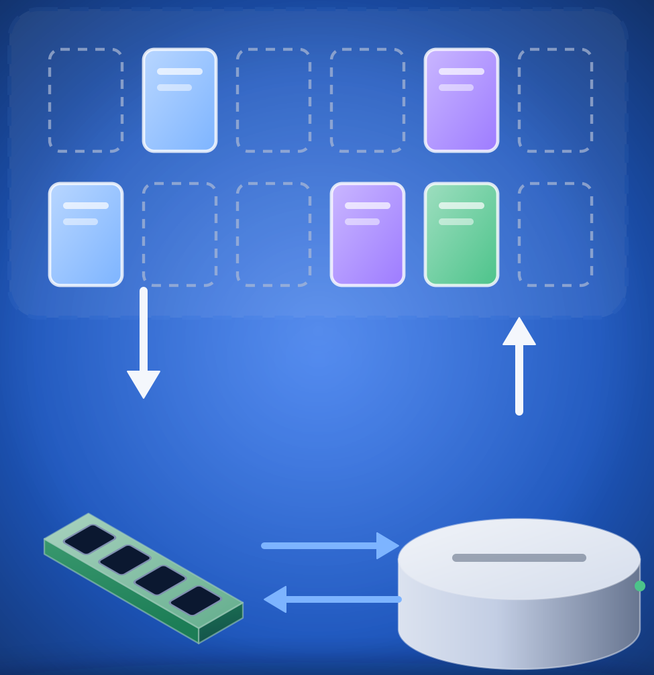
Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · WEEK 13

Virtual Memory

Running programs that are only partly in memory: demand paging, page replacement, frame allocation and thrashing.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Define virtual memory and describe its benefits.
2. Explain how pages are loaded into memory by demand paging.
3. Apply the FIFO, optimal and LRU page-replacement algorithms.
4. Describe the working set of a process and how it relates to locality.
5. Explain thrashing and how to limit it.
6. Describe how Linux, Windows and Solaris manage virtual memory.

THIS WEEK

Topic 11

Week 12 placed whole programs in memory. Most of a program is not needed most of the time.

DISCUSSION

A laptop with 8 GB runs programs that need 20 GB

A browser, an editor, a virtual machine and a game, all open. Nothing crashes. Where is the rest?

Week 13

PART 1

Background

The logical memory a program sees is separated from the physical memory the machine has.

Programs do not need all of their memory

Rarely used code

Routines for unusual error conditions are almost never executed.

Oversized data

Arrays, lists and tables are often declared larger than they turn out to need.

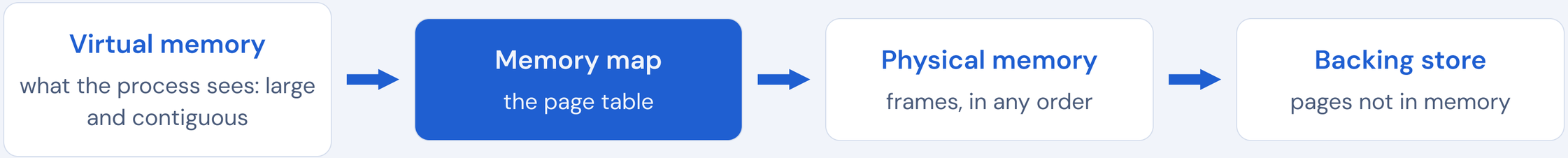
Not all at once

Even where the whole program is used, it is not all needed at the same time.

So load only part of it

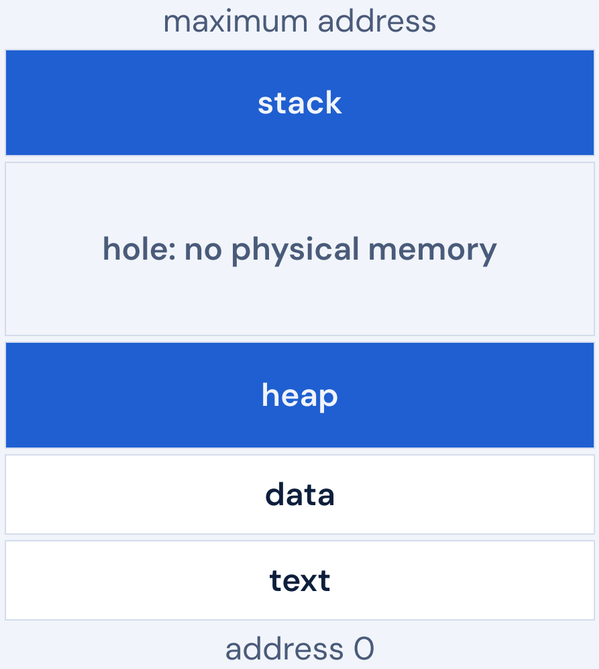
A program is no longer limited by physical memory. Each one uses less memory, so more can run together, and less I/O is needed to load or swap it.

Virtual memory



Only part of the program has to be in memory for it to run. The logical address space can be much larger than physical memory, and can be shared by several processes.

The virtual address space



The logical view of how a process is stored: usually from address 0, contiguous to the end of the space.

The stack grows down and the heap grows up. The hole between them needs physical pages only if they grow into it. Such a space is called sparse.

Pages can be shared: a library is mapped into many address spaces, and fork() can let parent and child share pages.

PART 2

Demand paging

Bring a page into memory only when it is needed.

Demand paging

A page is loaded only when the process *refers to it*.

Lazy

Like swapping with paging, but a page is never brought in unless it will be used. The component that does this is called the pager.

What it saves

Less I/O, less memory, a faster response, and room for more users.

What it needs

A way to tell the pages in memory from the pages on disk: the valid–invalid bit.

The valid–invalid bit

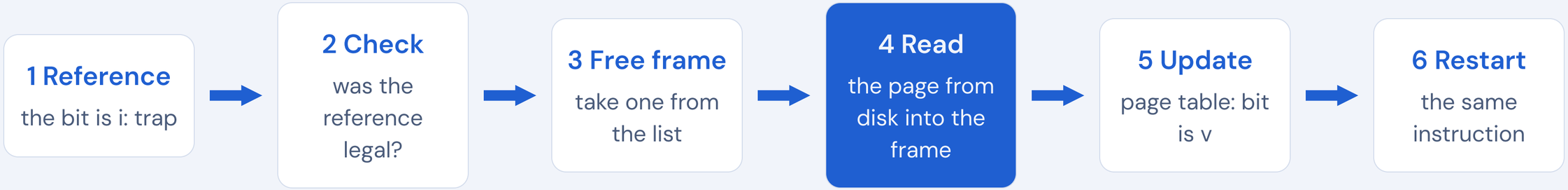
Page	Frame	Bit
0	4	v
1		i
2	6	v
3		i
4	9	v
5		i

Each page-table entry has a valid–invalid bit.

v: the page is legal and in memory. i: the page is not in memory, or is not part of the address space at all.

At the start every entry is i. When the MMU translates an address and finds i, it raises a page fault: a trap to the operating system.

Handling a page fault



If the reference was illegal, the process is ended. If it was legal, the page is simply not in memory yet. After step 6 the process continues as if the page had always been there.

Aspects of demand paging



Pure demand paging

Start a process with no pages in memory. The first instruction faults at once, and so does every first access to a page.



Locality of reference

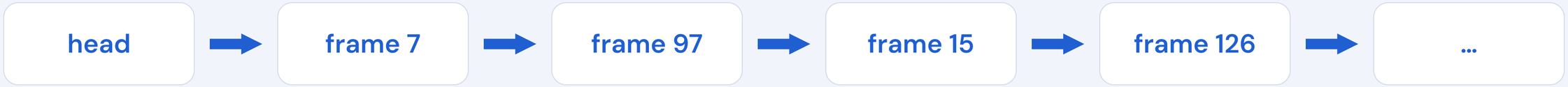
One instruction may touch several pages, for code and for data. In practice programs keep to a small set of pages at a time, which is what makes demand paging tolerable.



Hardware needed

A page table with the valid–invalid bit, secondary memory used as swap space, and the ability to restart an instruction.

The free-frame list



The operating system keeps a pool of free frames to satisfy page faults. Before a frame is given to a process its contents are cleared, zero-fill-on-demand, so one process never sees another's old data. At start-up all memory is on this list.

The performance of demand paging

$$EAT = (1 - p) \times ma + p \times \text{fault time}$$

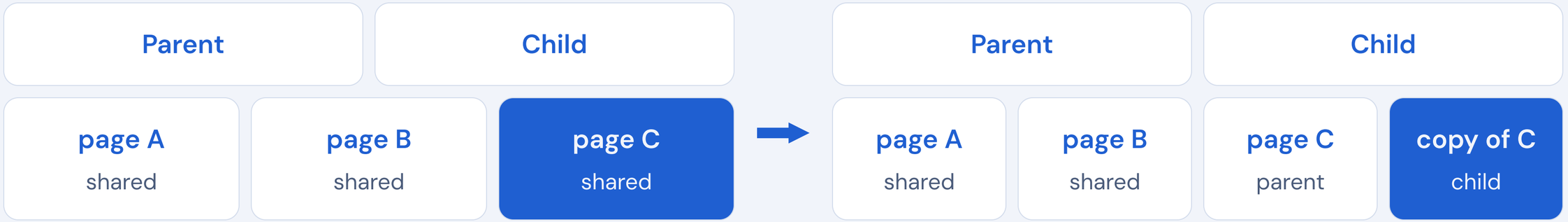
p is the page-fault rate, from 0 to 1. ma is the memory-access time. The fault time is dominated by reading the page from disk.

Take ma = 100 nanoseconds and a fault time of 10 milliseconds, which is 10,000,000 nanoseconds.

Fault rate p	EAT	Slowdown
0	100 ns	none
1 in 10,000	about 1,100 ns	11 times
1 in 1,000,000	about 110 ns	10%

One fault in ten thousand accesses makes the machine eleven times slower. To lose only 10%, fewer than one access in a million may fault.

Copy-on-write



After fork(): both processes point to the same pages.

The child writes to page C: only that page is copied.

Process creation is fast, because nothing is copied until it is modified. If the child calls exec() straight away, almost no copying is ever done.

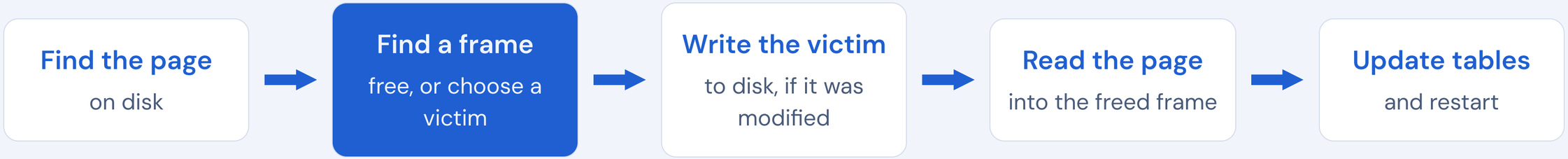
PART 3

Page replacement

A page fault arrives, and there is no free frame.

Week 13

Basic page replacement



The modify bit

Also called the dirty bit. Set when a page is written. Only modified victims need to be written back to disk.

Two decisions

A frame-allocation algorithm: how many frames each process gets. A page-replacement algorithm: which frame to replace.

The goal

The lowest page-fault rate, measured on a reference string: the sequence of page numbers a process refers to.

First-in, first-out

Reference	3	1	4	1	2	3	1	4	3	2	4	3
Frame 1	3	3	3	3	2	2	2	4	4	4	4	4
Frame 2		1	1	1	1	3	3	3	3	2	2	2
Frame 3			4	4	4	4	1	1	1	1	1	3
Page fault	F	F	F		F	F	F	F		F		F

Replace the page that has been in memory longest. Three frames, twelve references: 9 page faults. Simple, but the oldest page may be one that is used all the time.

Belady's anomaly

9

Faults with 3 frames

FIFO on the string 1 2 3 4 1 2 5 1 2 3 4 5.

10

Faults with 4 frames

The same string, the same algorithm, more memory.

We expect more frames to mean fewer faults. With FIFO, for some reference strings, adding a frame increases the number of faults.

The optimal algorithm

Reference	3	1	4	1	2	3	1	4	3	2	4	3
Frame 1	3	3	3	3	3	3	3	3	3	3	3	3
Frame 2		1	1	1	1	1	1	4	4	4	4	4
Frame 3			4	4	2	2	2	2	2	2	2	2
Page fault	F	F	F		F			F				

Replace the page that will not be used for the longest time. 5 page faults: no algorithm can do better on this string. It cannot be implemented, because it needs the future. We use it as the yardstick for the others.

Least recently used

Reference	3	1	4	1	2	3	1	4	3	2	4	3
Frame 1	3	3	3	3	2	2	2	4	4	4	4	4
Frame 2		1	1	1	1	1	1	1	1	2	2	2
Frame 3			4	4	4	3	3	3	3	3	3	3
Page fault	F	F	F		F	F		F		F		

Replace the page that has not been used for the longest time: the past as a guess at the future. 7 page faults, between FIFO and optimal. LRU does not suffer from Belady's anomaly.

Implementing LRU

A TIME FOR EACH PAGE

Counters

Every page-table entry has a time-of-use field, and the clock is copied into it at each reference. To replace, search for the smallest value.

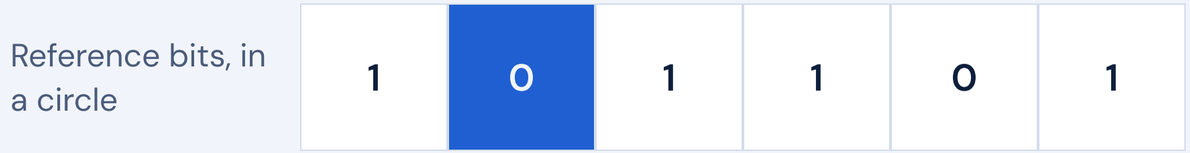
AN ORDER OF PAGES

A stack

Keep the page numbers in a doubly linked list. A referenced page moves to the top. The bottom is always the least recently used: no search, but every reference updates pointers.

Both need hardware help on every memory reference, which is too costly. Real systems approximate LRU.

Approximating LRU: second chance



The reference bit

One bit per page, set to 1 by the hardware whenever the page is used. We know which pages were used, not in what order.

The clock algorithm

FIFO with a second chance. A pointer moves around the frames. A page with bit 1 has its bit cleared and is passed over. The first page found with bit 0 is replaced.

Enhanced version

Use the pair (reference, modify). Prefer (0, 0): not used and not modified. Last choice (1, 1): used and modified.

Counting and page buffering



Counting algorithms

Keep a count of references to each page. Least frequently used replaces the smallest count. Most frequently used argues the opposite: a small count means the page has only just arrived. Neither is common.



Page buffering

Always keep a pool of free frames, so a fault never waits for a victim to be written out. Write modified pages to disk when the device is idle. Remember what each free frame held: if the page is needed again before reuse, no I/O is needed.

PART 4

Allocation of frames

How many frames does each process get?

Fixed allocation

THE SAME FOR ALL

Equal allocation

Every process gets the same number. With 90 frames and 2 processes, each gets 45, whatever its size.

BY SIZE

Proportional allocation

Frames in proportion to size. A process of 20 pages and one of 160 pages share 90 frames: 20/180 × 90 = 10 frames, and 160/180 × 90 = 80 frames.

Each process also needs a minimum number of frames, set by the architecture: enough for all the pages a single instruction can touch.

Global and local replacement

FROM ANYONE

Global

A process chooses a victim from all the frames, so it can take one from another process. Better throughput, and the more common choice. But a process's fault rate depends on what the others do.

FROM ITSELF

Local

A process chooses only among its own frames. Its performance is more consistent, but memory that other processes are not using cannot help it.

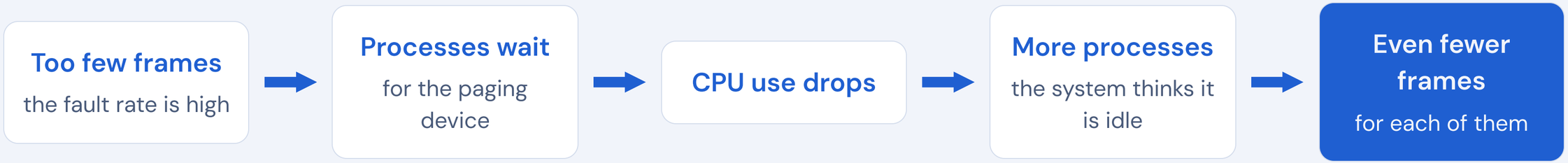
With global replacement, a kernel routine reclaims pages when free memory falls below a minimum threshold, and stops when it climbs back above a maximum threshold.

PART 5

Thrashing

A process that spends more time paging than executing.

How thrashing begins



A vicious circle. Each new process takes frames from the others, which fault more, which lowers CPU utilization further. Throughput collapses.

Locality

A *locality* is a set of pages that are actively used together.

Why paging works

A process moves from one locality to another as it runs. While it stays in one, it faults little.

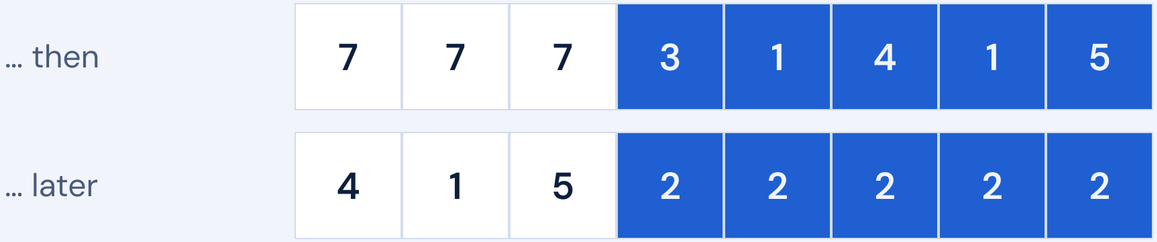
Why thrashing occurs

The sum of the sizes of all localities is larger than total memory.

What to do

Give each process enough frames for its current locality, and no more.

The working-set model



The window Δ

A fixed number of the most recent page references. Here Δ = 5, shown highlighted.

The working set

The set of pages in the window. First it is {1, 3, 4, 5}: size 4. Later it is {2}: size 1.

The policy

D is the sum of all working-set sizes. If D is greater than the number of frames, suspend a process. Otherwise another may start.

Page-fault frequency

Above the upper bound

the process needs frames: give it one

Between the bounds

acceptable: leave it alone

Below the lower bound

it has too many: take one away

A more direct approach than the working set.

Set an acceptable range for the page-fault rate of a process, and adjust its number of frames to keep it there.

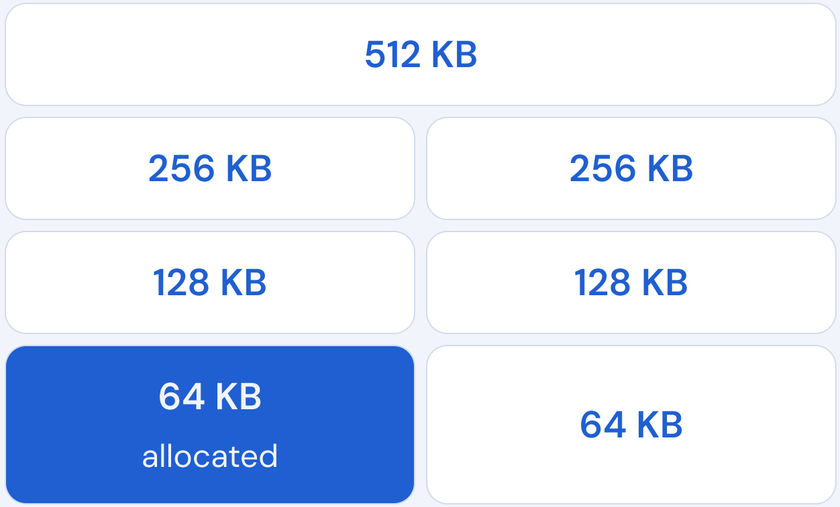
If the rate is too high and no frames are free, a process is suspended and its frames are shared out.

PART 6

Kernel memory and other considerations

Memory for the kernel itself, and the design choices around paging.

Allocating kernel memory



Buddy system: a request for 40 KB from a 512 KB segment.

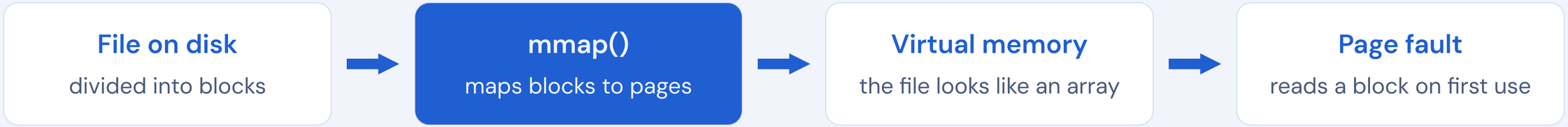
Buddy system

Memory is split into halves, called buddies, until a block just large enough is reached. Sizes are powers of 2. Free buddies merge again quickly. The cost is internal fragmentation: here 24 KB.

Slab allocation

A cache for each kind of kernel object, filled with objects ready to use. No fragmentation, and a request is met at once. Linux uses it.

Memory-mapped files



A file is treated as part of the virtual address space. Reading and writing it become ordinary memory accesses, with no read() or write() system call each time. Several processes can map the same file and so share memory.

Other considerations

Prepaging

Bring in the pages a process will need before it asks, to avoid a burst of faults at the start. Worth it only if most of them are used.

Page size

Small pages: less fragmentation, better locality. Large pages: smaller tables, less I/O, more TLB reach.

TLB reach

The memory the TLB can map: entries × page size. 64 entries of 4 KB reach 256 KB. With 2 MB pages they reach 128 MB.

Program structure

A loop that walks a large matrix along its rows may fault far less than one that walks along its columns.

I/O interlock

A page being used for I/O must be locked in memory, so it is not chosen as a victim.

Memory compression

Compress several pages into one frame instead of writing them out. Used on mobile systems, which do not swap.

Operating-system examples

LISTS

Linux

Demand paging with global replacement similar to the clock algorithm. Pages are kept on an active list and an inactive list. A kernel routine reclaims pages when free memory is low.

WORKING SETS

Windows

Demand paging with clustering: the faulting page is brought in together with its neighbours. Each process has a working-set minimum and maximum, trimmed when memory is short.

A SCANNER

Solaris

A page scanner runs when free memory falls below a threshold, using a clock algorithm. The less free memory there is, the faster it scans.

Key takeaways

01

Virtual memory

A process sees a large private address space. Only the pages in use are in physical memory.

02

Demand paging

A page is loaded at the first reference to it. A page fault is expensive, so faults must be rare.

03

Page replacement

FIFO is simple, optimal is the yardstick, and LRU and its approximations are what systems use.

04

Thrashing

Too few frames for the localities in use. The working set and page-fault frequency keep it away.

Next session

COMING UP

Review

Week 14 reviews Topics 6 to 11: synchronization, deadlock and memory management. The final examination follows.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 10.